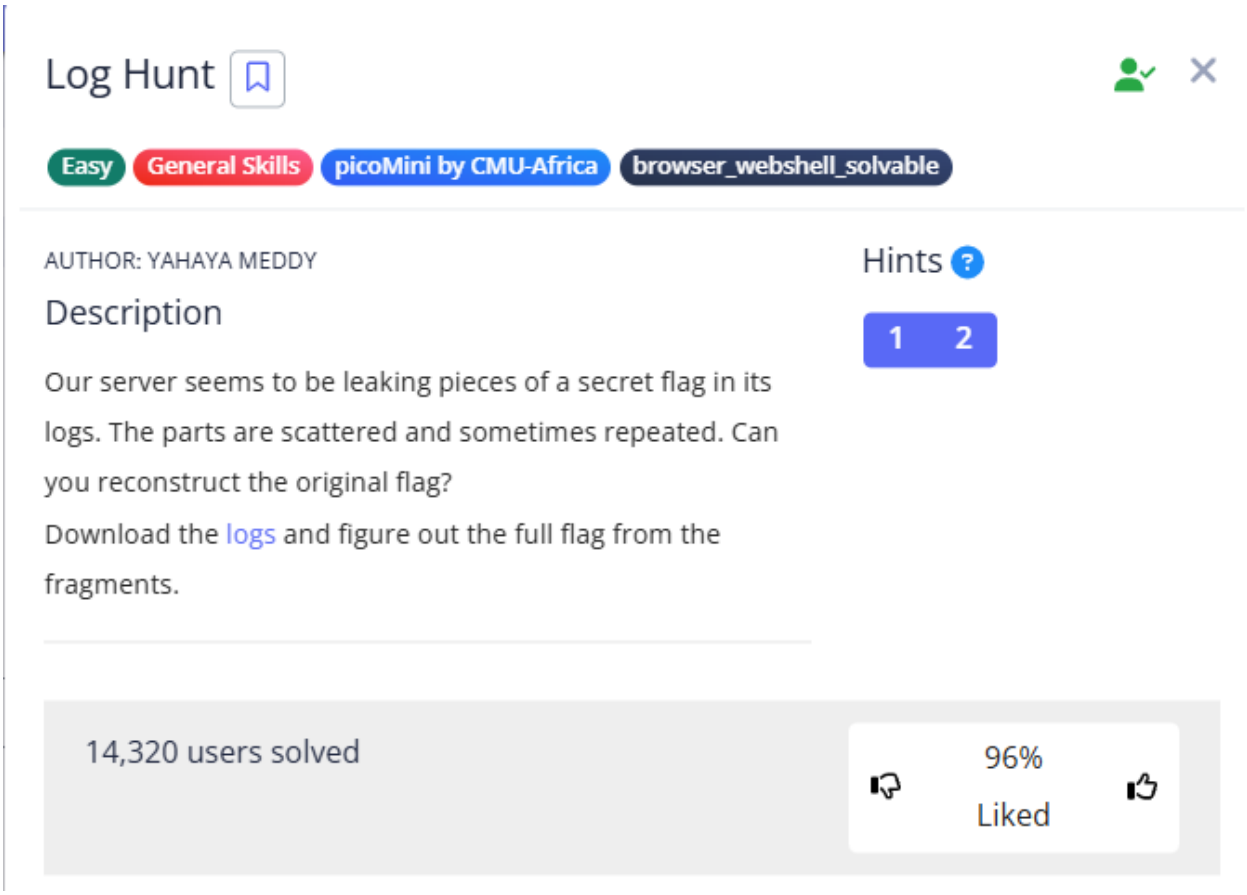
Aymen Abdullah Mohammed Eldoumah

PicoCTF Challenge Report: Log Hunt (General Skills)

Challenge: Log Hunt

Category: General Skills

Objective: To reconstruct a fragmented and scattered secret flag from a large server log file (logs).



Key Findings and Methodology

The challenge required isolating specific flag segments from a heavily polluted and repetitive log file.

1. Initial Discovery

Initial inspection and filtering using the standard flag pattern proved difficult due to excessive repetition and fragmentation:

cat logs | grep picoCTF

The key insight was noticing that all relevant flag segments were consistently prefaced by the identifier **flagpart**. This string served as a unique marker for all pieces of the secret.

2. Isolation and Cleaning

To efficiently extract the unique fragments, the filtering strategy was shifted to target the discovered identifier (flagpart), combined with standard Linux text utilities to clean the output.

The optimal command pipeline was:

Bash

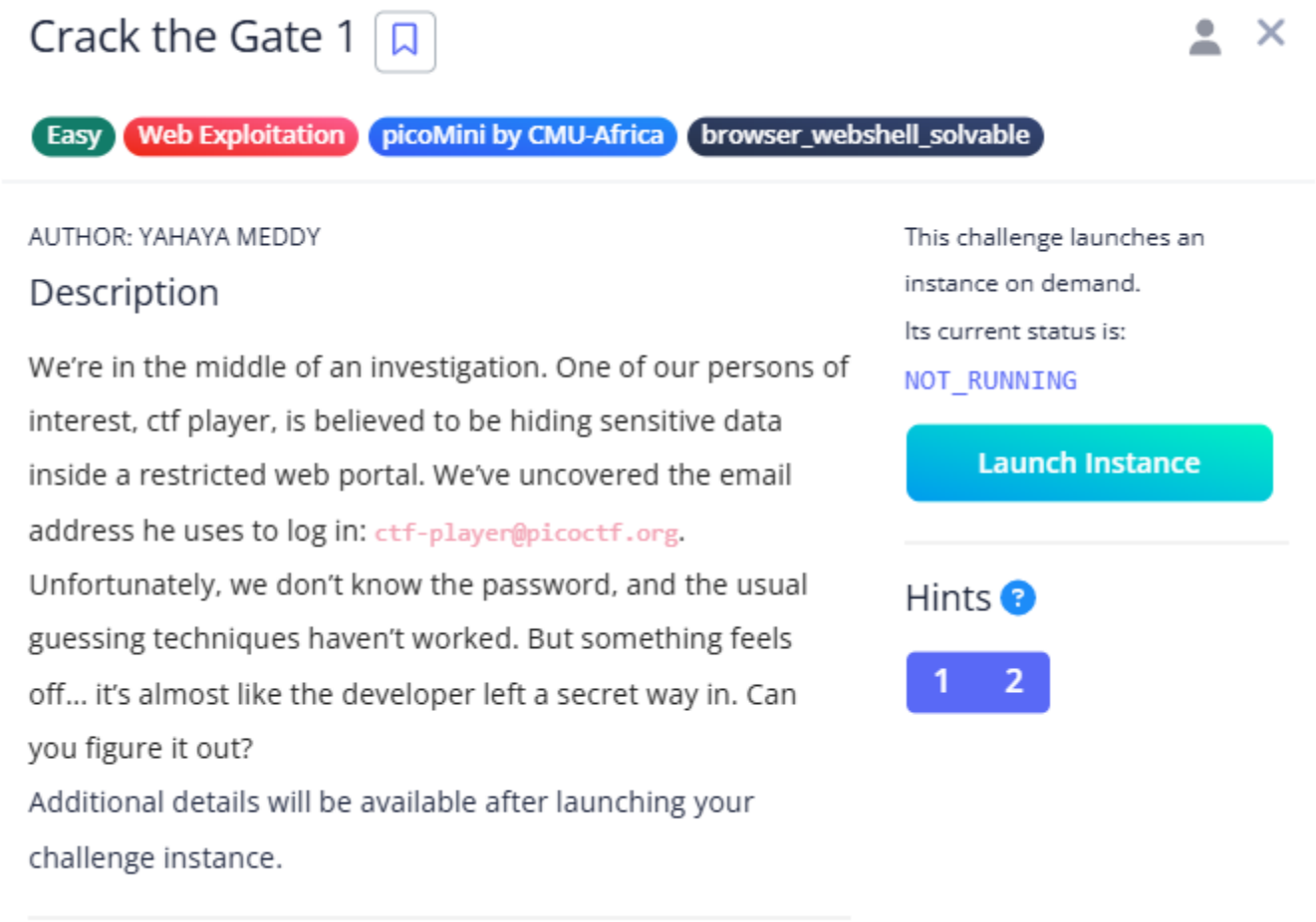
```
cat logs | grep flagpart | sort | uniq
```

Command	Purpose
grep flagpart	Filters the log file to show only lines containing the unique identifier flagpart.
sort	Arranges the resulting lines alphabetically/numerically, grouping duplicates together.
uniq	Removes all duplicate lines, leaving only the unique, non-repeated flag fragments.

3. Conclusion

By isolating the unique fragments using the flagpart identifier and the sort | uniq pipeline, the pieces were successfully extracted and reconstructed into the final, complete flag.

Challenge Report: Crack the Gate 1 (Web Exploitation)



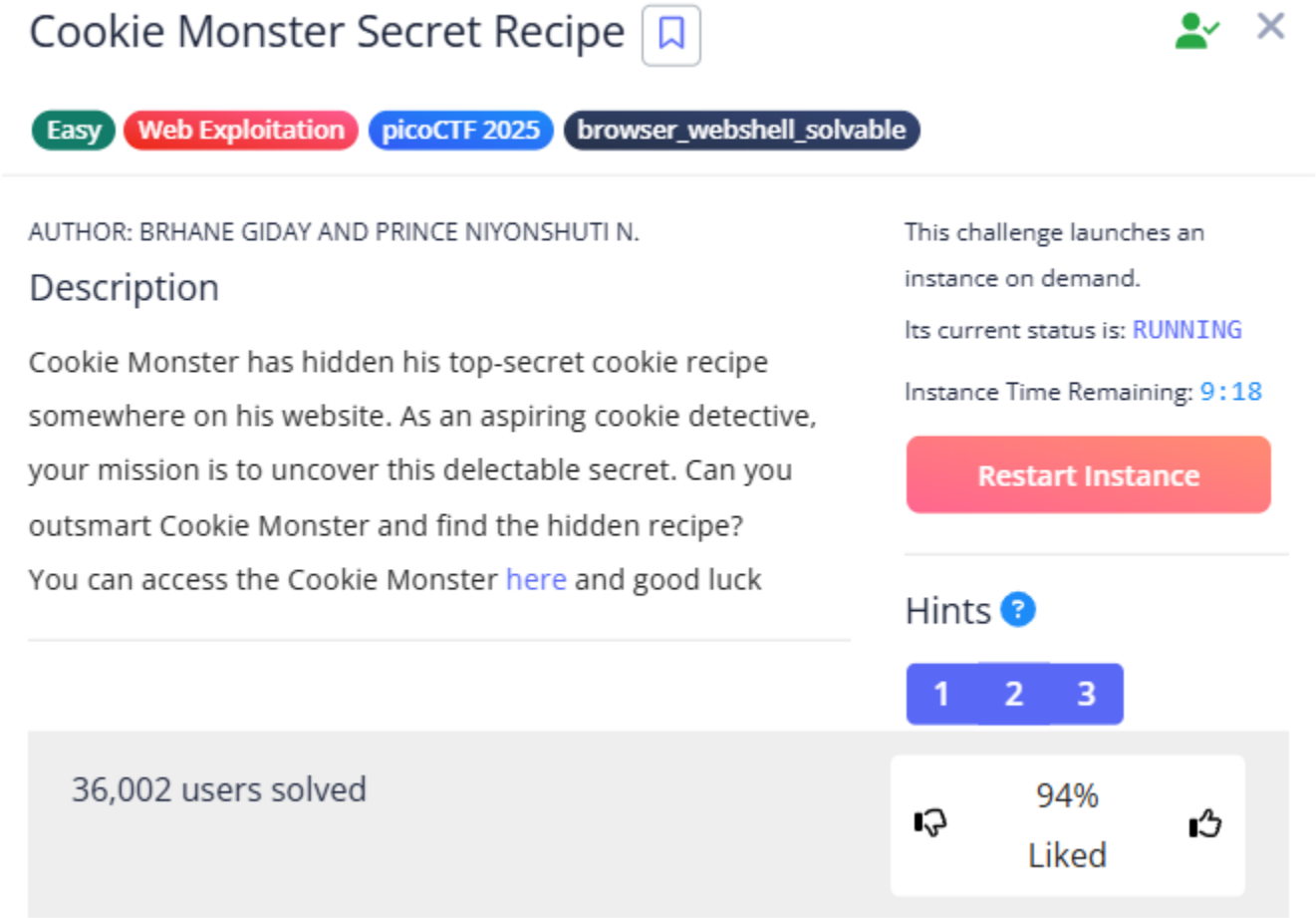
This challenge required bypassing a login gate using a known email (ctf-player@picoctf.org) without knowing the password.

Key Methodology and Steps

- **Initial Code Analysis (Source Code Inspection):**
  - The application's **Source Code** was meticulously inspected (View Page Source).
  - A **hidden comment** was discovered within the code's structure.
- **Cryptographic Discovery (ROT13):**
  - The discovered comment was identified as being encoded using the **ROT13** algorithm.
  - Decrypting the comment revealed **crucial information** pointing to a specific, non-standard **HTTP Header.**

- **Network Analysis and Request Capture:**
    - A simulated login attempt was performed using a random/dummy password to capture the full **HTTP Request** transmitted to the server (via Developer Tools Network tab).
    - The complete request was copied as a **cURL** command.
- **Exploitation via Shell:**
    - The **cURL** command was executed in the **Shell.**
    - The secret **Header**, derived from the ROT13 decryption, was manually added to the cURL command.
- **Conclusion:**
    - By including the required secret HTTP Header in the request, the login validation was successfully bypassed.
    - The **Flag** was received directly within the server's response, completing the challenge.

Challenge Report: Cookie Monster Secret Recipe (Web Exploitation)



Objective: Discover the hidden cookie recipe (the Flag) on the website.

Key Methodology and Steps (المنهجية والخطوات الأساسية)

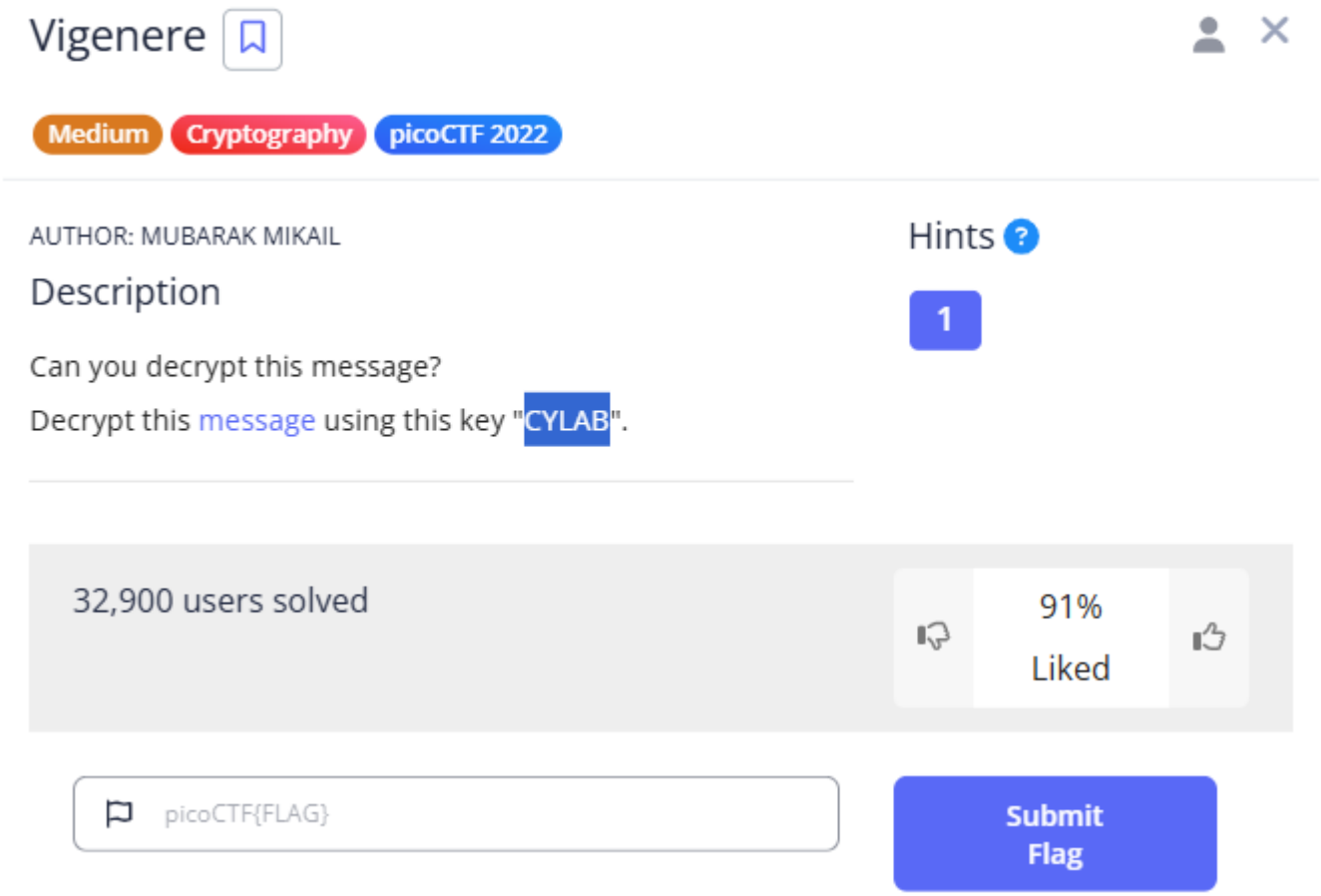
- **Initial Reconnaissance & Hint Acquisition:**
  - Arbitrary data (random values) was submitted to the website input field.
  - The server responded with a crucial hint message: **"Hint: Have you checked your cookies lately?"**
- **Cookie Inspection:**
  - The **Developer Tools** were opened, and the **Application** (Storage) tab was accessed to inspect the browser's cookies.
  - A suspicious cookie, likely named after the recipe or a secret, was identified.

    - The value of this cookie was a long, non-standard string of characters.

- **Decoding the Secret:**
    - The cryptic string was recognized as an **encoded value**.
    - Based on its character set (alphanumeric characters, /, +, and potential padding =), it was identified as **Base64 encoding**.
    - The cookie value was then decoded from **Base64**.

- **Conclusion:**
    - The result of the Base64 decoding revealed the hidden recipe, which was the final **Flag**. The challenge relied on following the server's explicit hint and understanding standard web encoding techniques.

Challenge Report: Vigenere (Cryptography)



1. The Quick and Simple Solution (CyberChef)

This was my fast-track approach to immediately applying the given key:

- **1. Extracting the Ciphertext:** I obtained the encoded text from the challenge file.
- **2. Receiving the Key:** I noted that the challenge directly provided the required key, which was **CYLAB**.
- **3. Using CyberChef:** I immediately used the CyberChef tool for efficiency.
- **4. Direct Application:** I input the ciphertext and specified the **Vigenere Decode** operation, entering the key **CYLAB** into the designated field.
- **5. Result:** The decryption was instantaneous, and I successfully extracted the final Flag.

2. Detailed Python Script Analysis (Demonstrating My Understanding)

After completing the challenge, I wrote my own Python code to prove my understanding of the cipher's inner workings, rather than relying solely on pre-built tools.

Variable	Value	Detailed Explanation (My Perspective)
instr	Ciphertext	This is the exact encoded text I copied from the challenge.
key	"CYLAB"	This is the explicit key I used for the operation.
dicts_l / dicts_u	Dictionaries	This is the core logic. I manually **built a map** for every character in the key (e.g., 'C') and every possible ciphertext character. This allows me to know instantly **(Cipher Character : Original Character)**, avoiding repeated calculations.
count	0	I used this counter to ensure I **cyclically transition** through the key letters (C, Y, L, A, B, C, Y, ...) as I process the characters of the encrypted text.

Analysis of the Decryption Loop

1. **Iteration:** I looped through every character in the ciphertext.
2. **Key Determination:** I used the **modulo operator** (count % len(key)) to accurately determine which key character to use for that specific position.
3. **Lookup:** I used the pre-built dictionaries (dicts_l or dicts_u) to look up the original character corresponding to the current ciphertext character and key combination.
4. **Result:** The final decoded letters were compiled into the **outstr** variable, successfully revealing the Flag.

```python
import string
instr="rgnoDVD{O0NU_WQ3_G1G3O3T3_A1AH3S_2951c89f}"
key="CYLAB"
outstr=""
offset_lower=ord('a')
offset_upper=ord('A')
lower=string.ascii_lowercase
upper=string.ascii_uppercase


dicts_l={}
for k in key:
    key_loc=ord(k)-offset_upper
    dicts_l[k]={}
    for l in lower:
        cipher=chr((ord(l)-offset_lower+key_loc)%26+offset_lower)
        dicts_l[k][cipher]=l #dicts_l[key][cipher]=[plain]


dicts_u = {}
for k in key:
    key_loc = ord(k) - offset_upper
    dicts_u[k] = {}
    for u in upper:
        dicts_u[k][chr((ord(u) - offset_upper + key_loc) % 26 + offset_upper)] = u # dicts_u[key][cipher]=[plain]

count=0
for c in instr:
    if c.isalpha():
        if c.islower():
            outstr+=dicts_l[key[count%len(key)]][c]
        elif c.isupper():
            outstr+=dicts_u[key[count%len(key)]][c]
        count+=1
    else:
        outstr+=c
print(outstr)
```